プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

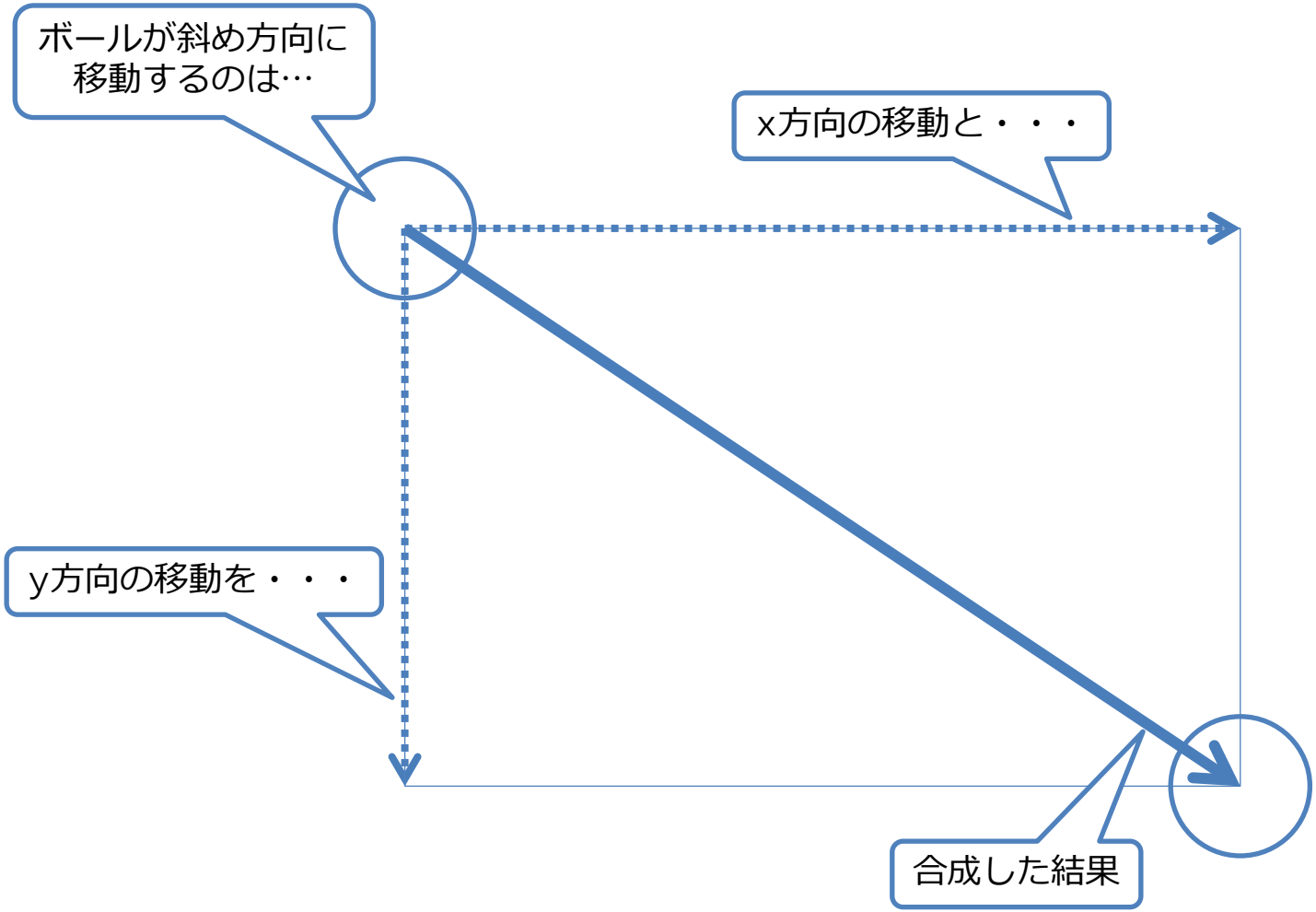
目次

- テーマ：ブロック崩し
- 例題：ボールが上下左右に反射する
- 課題：ボールが上下左右に反射する

ボールが上下左右に反射する

- 前回(第2回)のプログラムを修正し、ボールが上下左右に反射するプログラムを作成する
- ボールが斜め方向に移動するためには、縦・横方向に同時に移動すると考える
  - x方向とy方向に同時に移動

ボールが斜め方向に移動する



プログラム例

```
int radius = 40;          // ボールの半径
float x = 110;            // ボールのx座標（初期値110）
float y = 60;             // ボールのy座標（初期値60）
float vx = 0.5;           // ボールのx方向の速度（初期値0.5）
float vy = 0.3;           // ボールのy方向の速度（初期値0.3）
int xdir = 1;             // ボールのxの進行方向（初期値1、-1:左方向、1:右方向）
int ydir = 1;             // ボールのyの進行方向（初期値1、-1:上方向、1:下方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS); // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);

  x += vx * xdir;         // ボールがx方向に移動する
  y += vy * ydir;         // ボールがy方向に移動する
```

プログラム例

```
  if ((x > width-radius) || (x < radius)) {    // 右壁か左壁に衝突する場合
    xdir = -xdir;                              // xの進行方向を反転する
  }

  if ((y > height-radius) || (y < radius)) {   // 下壁か上壁に衝突する場合
    ydir = -ydir;                              // yの進行方向を反転する
  }

  ellipse(x, y, radius, radius);               // ボールを描く
}
```

関数を使って書き直す

- 前回同様関数化する
  - ボールが移動する　`void move()`
  - ボールが右壁に衝突するかどうか　`boolean isHitRightWall()`
  - ボールが左壁に衝突するかどうか　`boolean isHitLeftWall()`
  - ボールが下壁に衝突するかどうか　`boolean isHitBottomWall()`
  - ボールが上壁に衝突するかどうか　`boolean isHitTopWall()`
  - ボールのxの進行方向を反転する　`void flipXdir()`
  - ボールのyの進行方向を反転する　`void flipYdir()`
  - ボールを描く　`void display()`

ボールが移動する

```
void draw() {
  background(0);
  x += vx * xdir;
  y += vy * ydir;
  if ((x > width-radius) ||
                  (x < radius)) {
    xdir = -xdir;
  }
  if ((y > height-radius) ||
                  (y < radius)) {
    ydir = -ydir;
  }

  ellipse(x, y, radius, radius);
}
```

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                  (x < radius)) {
    xdir = -xdir;
  }
  if ((y > height-radius) ||
                  (y < radius)) {
    ydir = -ydir;
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する
void move(){
  x += vx * xdir;
  y += vy * ydir;
}
```

関数の呼出

関数の定義

ボールのxの進行方向を反転する

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    xdir = -xdir;
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    ydir = -ydir;
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する
void move(){
  x += vx * xdir;
  y += vy * ydir;
}
```

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    flipXdir();
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    ydir = -ydir;
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する（省略）
// ボールのxの進行方向を反転する
void flipXdir(){
  xdir = -xdir;
}
```

関数の呼出

関数の定義

ボールのyの進行方向を反転する

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    flipXdir();
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    ydir = -ydir;
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する（省略）
// ボールのxの進行方向を反転する
void flipXdir(){
  xdir = -xdir;
}
```

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    flipXdir();
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    flipYdir();
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する（省略）
// ボールのxの進行方向を反転する（省略）
// ボールのyの進行方向を反転する
void flipYdir(){
  ydir = -ydir;
}
```

関数の呼出

関数の定義

ボールを描く

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    flipXdir();
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    flipYdir();
  }

  ellipse(x, y, radius, radius);
}

// ボールが移動する（省略）
// ボールのxの進行方向を反転する（省略）
// ボールのyの進行方向を反転する
void flipYdir(){
  ydir = -ydir;
}
```

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                    (x < radius)) {
    flipXdir();
  }
  if ((y > height-radius) ||
                    (y < radius)) {
    flipYdir();
  }

  display();
}
// ボールが移動する（省略）
// ボールのxの進行方向を反転する（省略）
// ボールのyの進行方向を反
// ボールを描く
void display(){
  ellipse(x, y, radius, radius);
}
```

関数の呼出

関数の定義

上下左右の壁に衝突したかどうか

- 課題　kadai02
- 例題を参考に、ボールが上下左右で反射するプログラムを完成させる

- 次回の講義では、kadai02を使います

プログラム例　変更後　kadai02

```
int radius = 40;          // ボールの半径
float x = 110;            // ボールのx座標（初期値110）
float y = 60;             // ボールのy座標（初期値60）
float vx = 0.5;           // ボールのx方向の速度（初期値0.5）
float vy = 0.3;           // ボールのy方向の速度（初期値0.3）
int xdir = 1;             // ボールのxの進行方向（初期値1、-1:左方向、1:右方向）
int ydir = 1;             // ボールのyの進行方向（初期値1、-1:上方向、1:下方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS); // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);
  move();                                          //ボールが移動する
  if (                  ||                  ) {    //右壁か左壁に衝突したら
    flipXdir();                                    //xの進行方向を反転する
  }
```

kadai02

```
  if (                    ||                    ) {        //下壁か上壁に衝突したら
    flipYdir();                                            //xの進行方向を反転する
  }
  display();                                               //ボールを描く
}

// ボールが移動する
void move(){
  x += vx * xdir;
  y += vy * ydir;
}

// ボールのxの進行方向を反転する
void flipXdir(){
  xdir = -xdir;
}
```

kadai02

```
// ボールのyの進行方向を反転する
void flipYdir(){
  ydir = -ydir;
}

// ボールを描く
void display(){
  ellipse(x, y, radius, radius);
}

// 下壁に衝突したかどうか
boolean isHitBottomWall(){

}
```

```
// 上壁に衝突したかどうか
boolean isHitTopWall(){

}

// 左壁に衝突したかどうか
boolean isHitLeftWall(){

}
// 右壁に衝突したかどうか
boolean isHitRightWall(){

}
```